

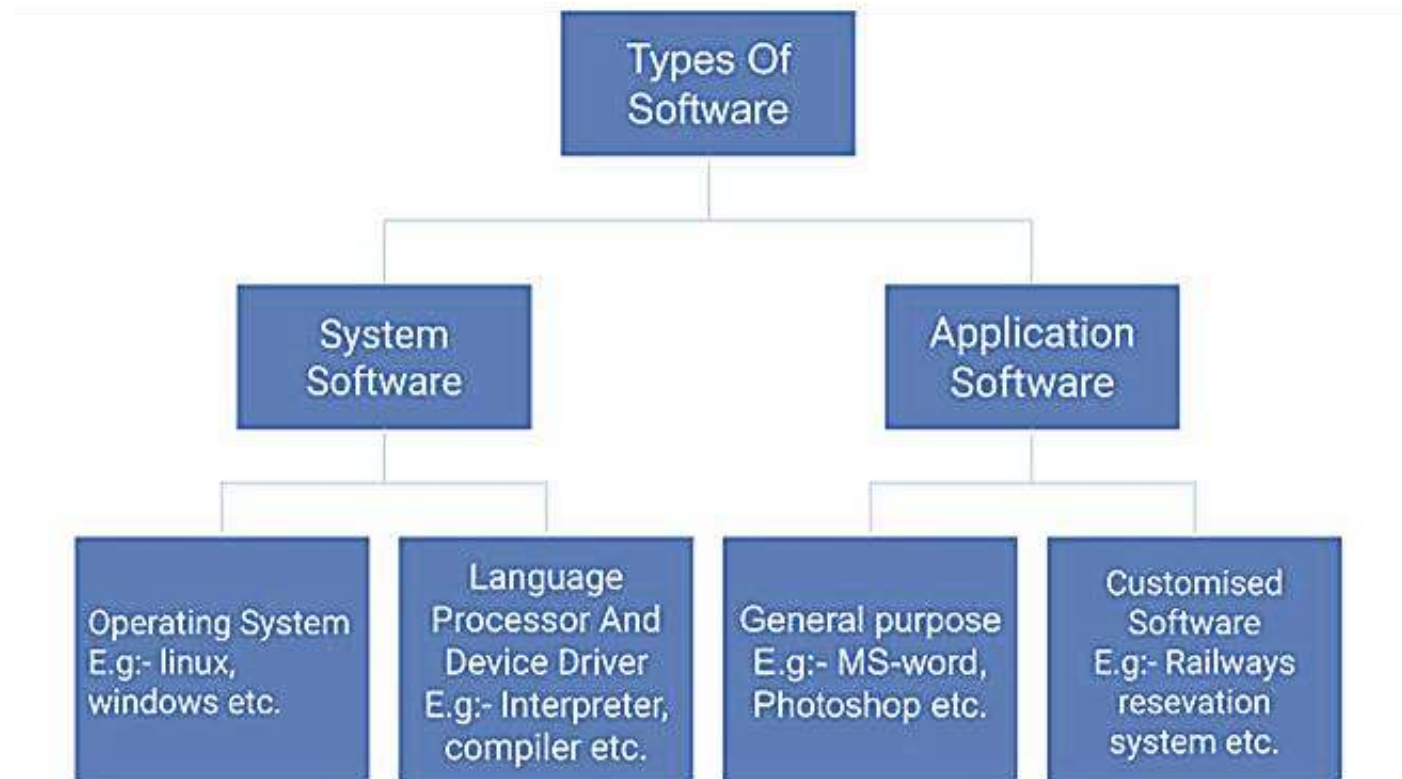
INTRODUCTION TO C

BY EKAGRA SIR

Syllabus: Introduction, Decision Control, Switch, Loops, Array & Pointers, Functions, Parameter Passing, Structure & Union, DMA, Macros, Miscellaneous

What is C?

- Programming Language (User-level)
- Procedural Oriented Language
- Combines High (User) + Low-level features



Life Cycle of a C Program

1. Preprocessor

1. Handles header files
2. Expands macros, includes files

2. Compilation

1. Syntactic Phase (Grammar check)
2. Semantic Phase (Logical mistakes)

Example: $Z = y / x;$ → DBZ (Divide by Zero error)

3. Loading (Loader): Loads program (prog.c) into RAM → Process

4. Linking (Linker): Connects functions, libraries (→ Procedural Oriented)

5. Runtime: Executes program and produces output

HEADER FILES

Preprocessor Directive in C

#include <stdio.h>

- **#include** → Preprocessor Directive
- **#** → Prevents re-compilation
- **stdio** → Provides Standard I/O functions: printf() and scanf()
- **.h** → Header File Extension

```
#include <stdio.h>
#include "Someheader.h"

int main() {
    -----
}
```

MAIN FUNCTION

Main Function in C

1. System-defined function
2. Entry point of a program
3. Provides resources to a program
4. There cannot be two entry points in any C program

Structure of main()

```
void main(argument_list)
{
    // body
}
```

The diagram illustrates the structure of the `main()` function. The word **Scopes** is written in red on the left. Two orange arrows originate from it: one points to the opening curly brace `{` and the other points to the closing curly brace `}` of the function body. The function signature `void main(argument_list)` and the body content `// body` are written in blue.

1. `main()`
2. `main(int argc, char *argv[])`
3. `main(int argc, char *argv[], char *env[])`

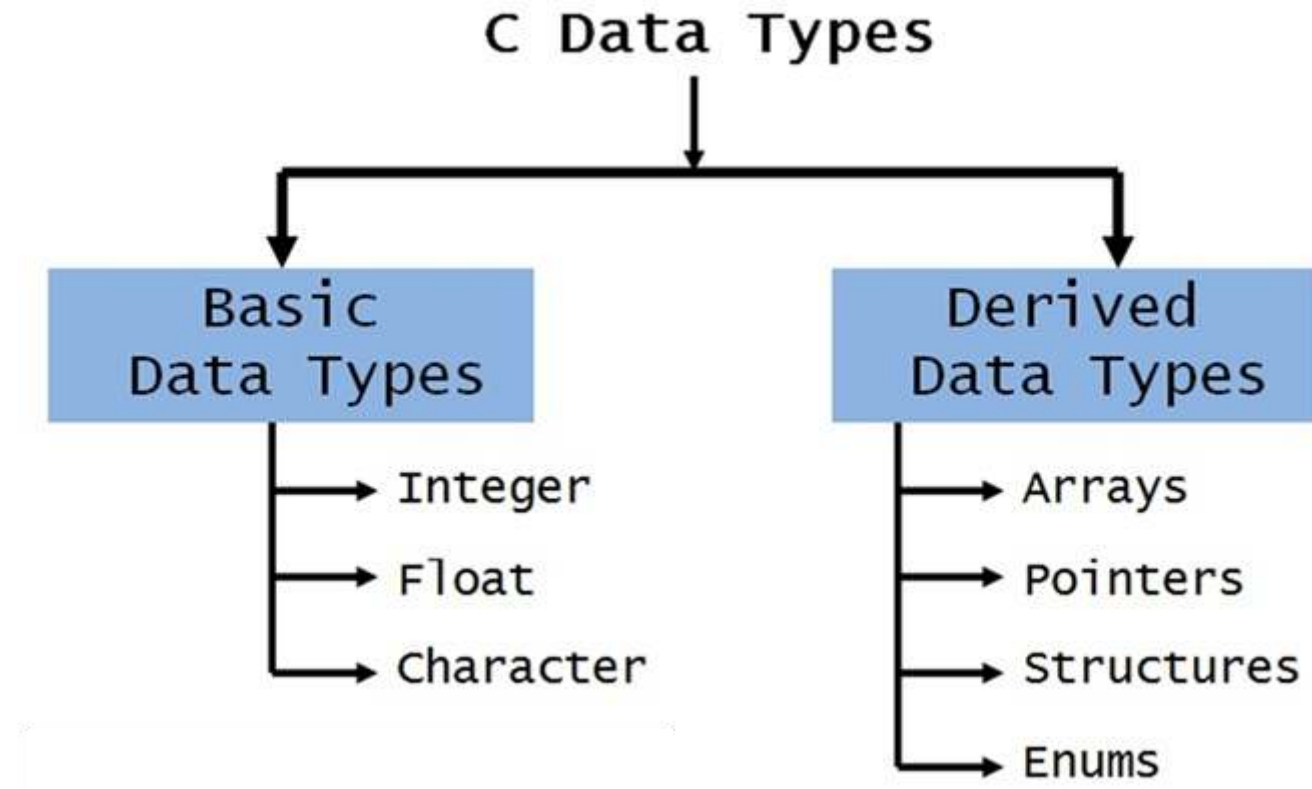
- `argc` → argument count (number of arguments).
- `argv` → argument vector (array of strings).
- `env` → environment variables (array of strings).
- Used for **command-line arguments**.

VARIABLES & DATATYPES

Variables are named memory locations used to store data.

OR

They create storage in memory for values.



A variable (datatype) has three parameters:

- * Size
- * Type
- * Range

Type	Typical Size	Range
int (16-bit CPU)	2 Bytes	-32,768 to +32,767
char	1 Byte	-128 to +127
float	4 Bytes	-3.4e-38 to +3.4e+38

For an n-bit signed datatype:

$$Range = -2^{n-1} \text{ to } +2^{n-1} - 1$$

where n = number of data lines (word length of CPU).

Important Point: Size of datatypes varies from compiler to compiler and machine to machine.

Rules for Naming Variables

- Must begin with a letter or underscore (_).
- Can contain letters, digits, and underscores.
- Case-sensitive (e.g., age and Age are different).
- Cannot use keywords (e.g., int, float, while).
- No special characters (@, #, \$, etc. not allowed).
- Should be meaningful and reflect the purpose.

Invalid Examples

- 123abc (cannot start with digit)
- total% (special character not allowed)
- float (keyword)

Valid Examples

- age, marks, _count, total1

```
main()
{
    int _ = 10;
    int __ = 30;
    int ____;

    ____ = _ + __;
    printf("%d", ____);
}
```

ASCII Code (*American Standard Code for Information Interchange*)

- A character encoding standard used in computers and communication devices.
- Each character (letters, digits, symbols) is represented by a numeric value (0–127).

ASCII Range

0 to 31 → Control characters (non-printable, e.g., newline, tab).

32 to 126 → Printable characters:

32 → Space

48–57 → Digits (0–9)

65–90 → Uppercase letters (A–Z)

97–122 → Lowercase letters (a–z)

127 → DEL (delete).

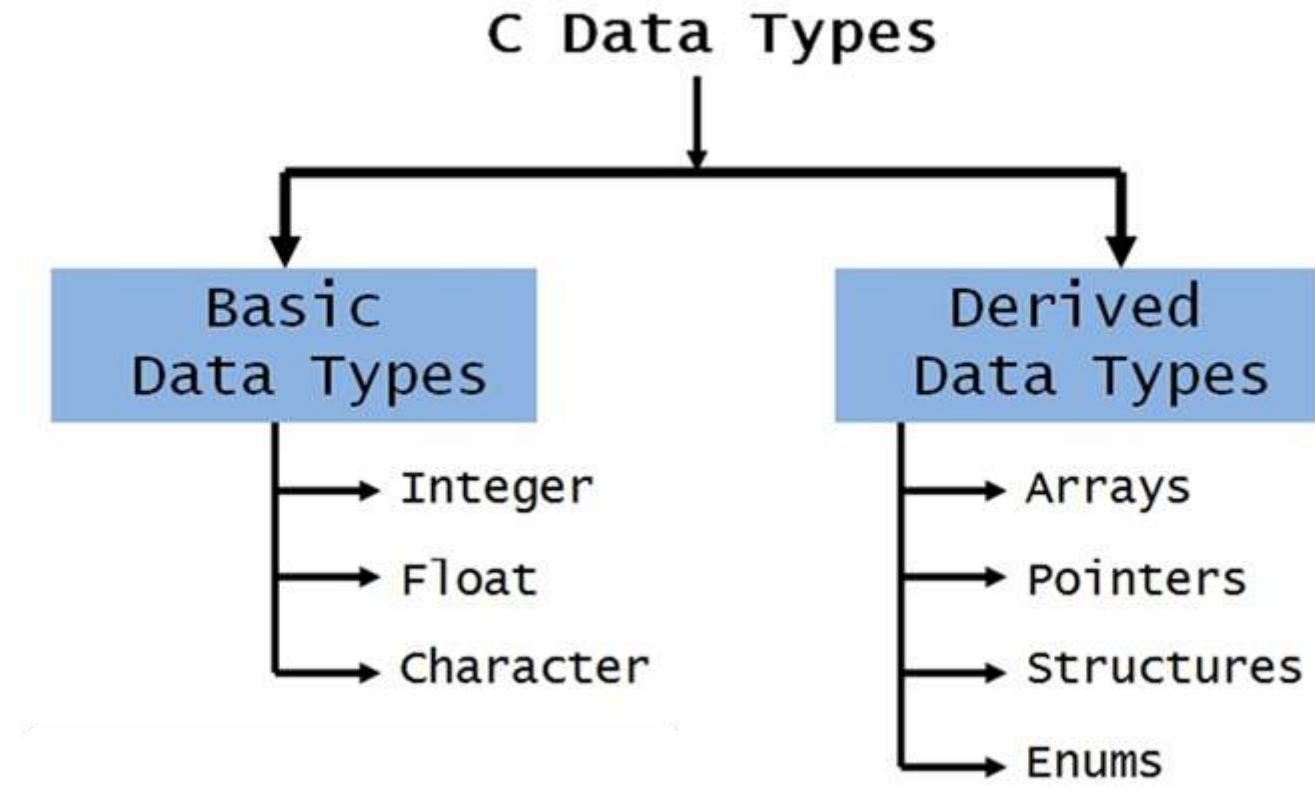
Character	ASCII Value
A	65
Z	90
a	97
z	122
0	48
9	57

VARIABLES & DATATYPES

Variables are named memory locations used to store data.

OR

They create storage in memory for values.



A variable (datatype) has three parameters:

- * Size
- * Type
- * Range

Type	Typical Size	Range
int (16-bit CPU)	2 Bytes	-32,768 to +32,767
char	1 Byte	-128 to +127
float	4 Bytes	-3.4e-38 to +3.4e+38

For an n-bit signed datatype:

$$Range = -2^{n-1} \text{ to } +2^{n-1} - 1$$

where n = number of data lines (word length of CPU).

Important Point: Size of datatypes varies from compiler to compiler and machine to machine.

Rules for Naming Variables

- Must begin with a letter or underscore (_).
- Can contain letters, digits, and underscores.
- Case-sensitive (e.g., age and Age are different).
- Cannot use keywords (e.g., int, float, while).
- No special characters (@, #, \$, etc. not allowed).
- Should be meaningful and reflect the purpose.

Invalid Examples

- 123abc (cannot start with digit)
- total% (special character not allowed)
- float (keyword)

Valid Examples

- age, marks, _count, total1

```
main()
{
    int _ = 10;
    int __ = 30;
    int ____;

    ____ = _ + __;
    printf("%d", ____);
}
```

ASCII Code (*American Standard Code for Information Interchange*)

- A character encoding standard used in computers and communication devices.
- Each character (letters, digits, symbols) is represented by a numeric value (0–127).

ASCII Range

0 to 31 → Control characters (non-printable, e.g., newline, tab).

32 to 126 → Printable characters:

32 → Space

48–57 → Digits (0–9)

65–90 → Uppercase letters (A–Z)

97–122 → Lowercase letters (a–z)

127 → DEL (delete).

Character	ASCII Value
A	65
Z	90
a	97
z	122
0	48
9	57

sizeof OPERATOR

- sizeof() operator is used to find the size (in bytes) of a datatype, variable, or expression.
- It is the only operator evaluated at compile time.

Syntax: sizeof (variable | type | value | expression)

Key Points

Expressions inside sizeof are not executed.

Their effect does not reflect outside the sizeof operator.

```
int a = 5;
```

```
int x = sizeof(a = 10); // expression not executed
```

```
printf("%d", a); // Output: 5
```

```
printf("%d", x); // Output: 2 (sizeof int which is 2 in 16-bit machine)
```

- Value of a remains unchanged (5).
- x stores the size of datatype int (e.g., 4 bytes on most systems).

COMMENTS

- **Comments** are non-executable statements in C.
- They are used to explain code and improve readability.
- Ignored by the compiler.

1. Single-line Comment

```
printf("Hello World"); // This is a single-line comment
```

2. Multi-line Comment

```
/* This is a  
multi-line comment */  
printf("Hello World");
```

Important:

1. Comments do not affect program execution.
2. Useful for Explaining logic, Debugging (temporarily disabling code), Documenting code for others

GARBAGE

Definition: When memory is assigned to a variable.

Example:

```
int x; // definition
```

```
int y = 10; // definition + initialization
```

Initialization: Assigning a specific value at the time of definition.

Example:

```
int y = 10; // initialized
```

```
int x; // uninitialized → garbage value
```

- Any unpredictable value is not initialization.

Garbage Value: If a variable is declared but not initialized, it holds a garbage value (random/unpredictable).

Example:

```
int x; // contains garbage value  
printf("%d", x);
```

extern Declaration

Example:

```
extern int z;
```

- This is only a declaration, not a definition.
- Memory will be assigned only if some function uses z.

Common Misconceptions

- “Compiler assigns garbage value” → Wrong statement
- Garbage value is just leftover memory content, not given by compiler.

Garbage in Java

In Java, default values are assigned by the garbage collector.

Example:

```
int x;
```

```
System.out.println(x); // 0
```

- Hence, unlike C, uninitialized variables in Java may default to 0 (for class-level variables).

INPUT & OUTPUT

- `printf()` is used to display output on the screen.
- It belongs to the `stdio.h` library

Syntax: `printf("Format String", ...);`

- **Format string** → a group of characters (character array).
- **Ellipsis** → Supports 0 or more arguments

Examples:

```
printf("Hello");    // prints Hello
```

```
printf("Hello %d", 10); // prints Hello 10
```

Key Points

Each `printf` statement must end with a semicolon (;) which is called statement.

Format Specifiers (in printf)

1. %4d

1. Print an integer (123456)
2. Field width = 4 → minimum 4 spaces reserved.
3. But since 123456 has 6 digits, it will print the whole number (ignores width if number is larger).

Output = 123456

2. %5.2f

2. Print a floating-point number (12131.13431)
3. Field width = 5 → minimum total width (but number can exceed it).
4. .2 → print exactly 2 digits after decimal.

Output = 12131.13

3. %3s

3. Print a string ("hello")
4. Field width = 3 → minimum 3 spaces.
5. But since "hello" has 5 characters, full string is printed.

Output = hello

Final Output (with spaces in between)

123456 12131.13 hello

Important Point:

- Width (4, 5, 3) = minimum characters reserved. If actual data is bigger, full value prints.
- 2 with %f = 2 decimal places.

- scanf() is used to take input from the user.
- Belongs to stdio.h library.

Syntax: scanf("Format Specifier", ...);

Ellipsis → Supports 0 or more arguments

```
int r1 = scanf("%d", &x);
```

```
// if input = 10 → r1 = 1 (1 successful input)
```

```
int r2 = scanf("%d %d", &x, &y);
```

```
// if input = 10 20 → r2 = 2 (2 successful inputs)
```

& → address operator (stores input at variable's memory address).

Function	Return Type	Return Value
printf()	int	Number of characters printed
scanf()	int	Number of successful inputs

Width in scanf

1. Strings (%s)

```
char str[20];  scanf("%4s", str);
```

- Input: knowledge
- Output stored in str: know
- Max 4 characters are read, rest ignored.

2. Integers (%d)

```
int x;  scanf("%3d", &x);
```

- Input: 12345
- scanf reads only 3 digits → x = 123
- The rest (45) stays in input buffer (can be read later).

3. Floats (%f)

```
float y;      scanf("%5f", &y);
```

- Input: 1234.56
- scanf reads at most 5 characters → 1234. (invalid float if cutoff badly).
- Often result = 1234.0

Key Difference from printf

1. **printf width** = *minimum* space reserved for output. (pads if smaller, prints full if bigger).
2. **scanf width** = *maximum* characters to read. (cuts input if longer).

```
10
11 int main()
12 {
13     char s[5];
14     int a;
15     float b;
16     scanf("%2d", &a);
17     scanf("%5f", &b);
18     scanf("%4s", s);
19
20
21     printf("%2d %3.2f %4s", a, b, s);
22     return 0;
23 }
```



```
1234 123.55 hellp
```

```
12 34.00 123.
```

OPERATORS & ITS TYPES

Description	Operator	Associativity
Function expression	()	Left to Right
Array Expression	[]	Left to Right
Structure operator	->	Left to Right
Structure operator	.	Left to Right
Unary minus	-	Right to left
Increment/Decrement	++ --	Right to Left
One's compliment	~	Right to left
Negation	!	Right to Left
Address of	&	Right to left
Value of address	*	Right to left
Type cast	(type)	Right to left
Size in bytes	sizeof	Right to left
Multiplication	*	Left to right
Division	/	Left to right
Modulus	%	Left to right
Addition	+	Left to right
Subtraction	-	Left to right
Left shift	<<	Left to right
Right shift	>>	Left to right
Less than	<	Left to right
Less than or equal to	<=	Left to right
Greater than	>	Left to right
Greater than or equal to	>=	Left to right
Equal to	==	Left to right
Not equal to	!=	Left to right

Description	Operator	Associativity
Bitwise AND	&	Left to right
Bitwise exclusive OR	^	Left to right
Bitwise inclusive OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	? :	Right to left
Assignment	=	Right to left
	*= /= %=	Right to left
	+= -= &=	Right to left
	^= =	Right to left
	<<= >>=	Right to left
Comma	,	Right to left

Operators are symbols that perform operations on operands.

- Important properties:
 - **Precedence** → Ranking of operators.
 - **Associativity** → Order of evaluation (Left-to-Right / Right-to-Left).

Example:

`int result = 10 + 3 * 4; // * has higher precedence than + result = 10 + 12 = 22`

Associativity Example:

`int x = 10, y = 20, z = 30;`

`int r = x > y > z; // evaluated Left-to-Right`

Important Points:

1. Every non-zero value is considered to be true where only 0 is treated as false.
2. Every non-zero value and 0 are complement of each other.

Type Conversion in Expressions

If operands are of different types, result is promoted to the type of the bigger operand.

Example:

```
int a = 10;
```

```
double b = 3.14;
```

```
double c = a + b; // result → double
```

Constant Expressions

Operators on constants or operands always yield a constant result except pointers

Examples:

3.14f + 3.14 → double

'a' + 'b' + 10 → converted to ASCII values → 97 + 98 + 10

Key Notes

- Conversion goes from smaller datatype → bigger datatype:

char → int → float → double

- Called Type Promotion (or Implicit Type Conversion).
- Explicit conversion can be done using Type Casting:

(int) 3.14 // result: 3

More Example:

area = b * h * ½ // result: 0

Reason: because ½ is treated as an integer expression, which evaluates to 0 instead of 0.5.

Hence correct expression would be...

area = b * h * (float) ½

Arithmetic Operators

Arithmetic operators are used to perform mathematical operations on numeric operands.
Commonly used in C programming for calculations.

Operator	Description	Example	Output
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	10 / 3	3
%	Modulus (remainder)	10 % 3	1

Key Points

- 1. Division of integers truncates the decimal part.
- 2. Modulus % works only with integers.
- 3. Can be used with variables and constants.

Unary Operators in C

- Unary operators act on a single operand to perform operations like increment, decrement, negation, or logical NOT. Few examples are given below:

Operator	Description	Example	Output
++x	Pre-increment	++a	a = a+1
x++	Post-increment	a++	Use then a = a+1
--x	Pre-decrement	--a	a = a-1
x--	Post-decrement	a--	Use then a = a-1
+	Unary plus (positive value)	+a	a
-	Unary minus (negation)	-a	-a
!	Logical NOT	!0	1
~	Bitwise NOT	~5	-6

Bitwise Operators in C

- Bitwise operators perform operations on individual bits of integer data types.
- Commonly used for masking, setting, clearing, and toggling bits.

Operator	Description	Example	Output
&	Bitwise AND	5 & 3	1
	Bitwise OR	5 3	7
^	Bitwise XOR (exclusive OR)	5 ^ 3	6
~	Bitwise NOT (one's complement)	~5	-6
<<	Left shift	5 << 1	10
>>	Right shift	5 >> 1	2

int a=3, b=5, c;

i) c = a;
a = b;
b = c;

ii) a = a + b;
b = a - b;
a = a - b;

iii) a = a * b;
b = a / b;
a = a / b;

iv) a = a^b;
b = a^b;
a = a^b;

v) a^ = b^ = a^ = b;

vi) b = (a + b) - (a = b);

Increment/Decrement Operators in C

- Increment operators are used to increase the value of a variable by 1.
- Two types:
 - Pre-Increment (++x)
 - Post-Increment (x++)

Pre-Increment (++x)

Increases the value of the variable before it is used in the expression.

Example:

```
int x = 5;
```

```
int y = ++x; // x is incremented first, then assigned to y
```

```
printf("%d %d", x, y); // Output: 6 6
```


Post-Increment (x++)

Uses the current value of the variable in the expression, then increments it.

Example:

```
int x = 5;

int y = x++; // y gets the current value of x, then x is incremented

printf("%d %d", x, y); // Output: 6 5
```

Feature	Pre-Increment (++x)	Post-Increment (x++)
Increment Time	Before use in expression	After use in expression
Returned Value	Incremented value	Original value
Usage	When you need incremented value immediately	When you need the original value first

```
1. main()
{
    int a =3,b;
    b=++a + a;
    printf("%d,%d",a,b);
}
```

```
2. main()
{
    int x=1,y=2,z;
    z=++x +x + y++;
    printf("%d,%d,%d",x,y,z);
}
```

```
3. main ()
{
    int i =10,j;
    j=i++ + ++i;
    printf("%d %d",i,j++);
}
```

```
4. main ()
{
    int a=5, b;
    b=++a +a + ++a;
    printf("%d%d",b,a);
}
```

5. main ()

```
{  
    int x=3,y=1;  
    ++y = ++x;  
    printf("%d %d",x,y);  
}
```

6. main()

```
{  
    int i=10,j;  
    j=i+++++i;  
    printf("%d",j);  
}
```

7. void main()

```
{  
    int i=-4,j;  
    j=i+++i;  
    printf("%d, %d",i-- ,j);  
}
```

8. void main()

```
{  
    int a=6, b;  
    b=+++a++;  
    printf("%d",b);  
}
```

Logical Operators in C

- Logical operators are used to combine or modify Boolean expressions.
- Result is either true (1) or false (0).
- Commonly used in conditional statements (if, while, for).

Operator	Description	Example	Output
&&	Logical AND	1 && 0	0
	Logical OR	1 0	1
!	Logical NOT (negation)	!1	0

Key Points

- && (AND): True if both operands are true.
- || (OR): True if any one operand is true.
- ! (NOT): Inverts the value of the operand.
- Logical operators short-circuit evaluation:
 - && stops at first false
 - || stops at first true

1. main()

```
{  
    int a = 2, b;  
    b = a && !a;  
    printf("%d", b);  
}
```

2. main()

```
{  
    int x = -1, y = 3, z;  
    z = (x < y) || (y == -3);  
    printf("%d", z);  
}
```

3. main()

```
{  
    int i = 3, j = 3, k;  
    k = (i != j) && (!i < j);  
    printf("%d", k);  
}
```

4. main()

```
{  
    int a = 1, b = 2, c;  
    c = (!a * 2) || (!b - 3);  
    printf("%d", c);  
}
```

5. main()

```
{  
    int x = 1, y = 5, z;  
    z = (x > 1) && (x < y) && !y;  
    printf("%d", z);  
}
```

6. main()

```
{  
    int i = 5, j = 10, k;  
    k = (i == 5) || (i > 10) && (i);  
    printf("%d", k);  
}
```

7. main()

```
{  
    int a = -1, b = 2, c;  
    c = ++a && ++b;  
    printf("%d%d%d", a, b, c);  
}
```

8. main()

```
{  
    int a = 0, b = 2, c;  
    c = ++a || ++b;  
    printf("%d%d%d", a, b, c);  
}
```

INTEGRAL DATA TYPES (IDTs)

Integral Data Types in C

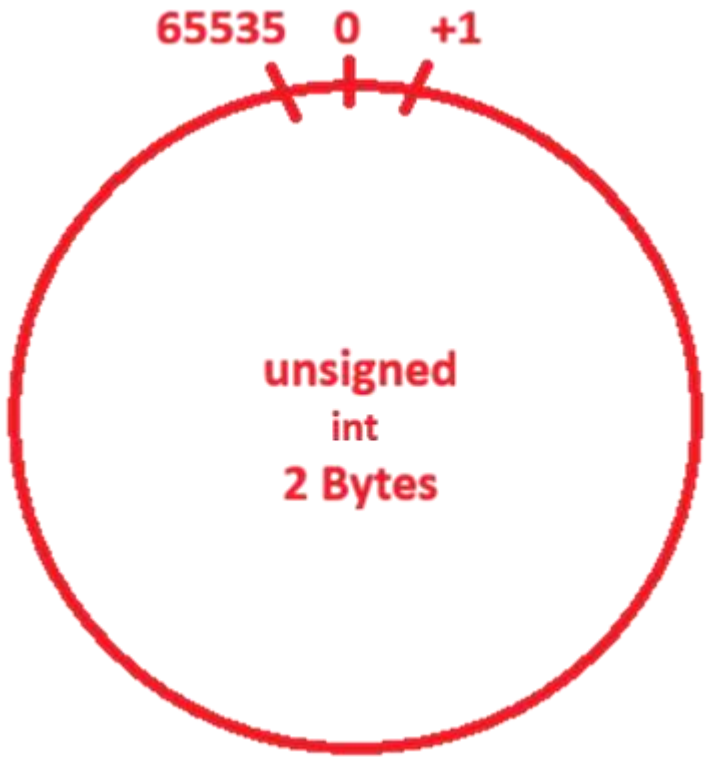
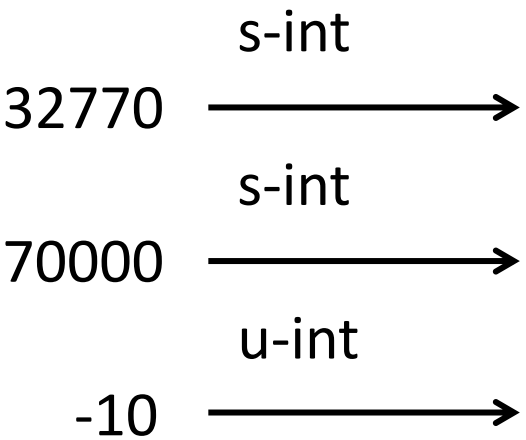
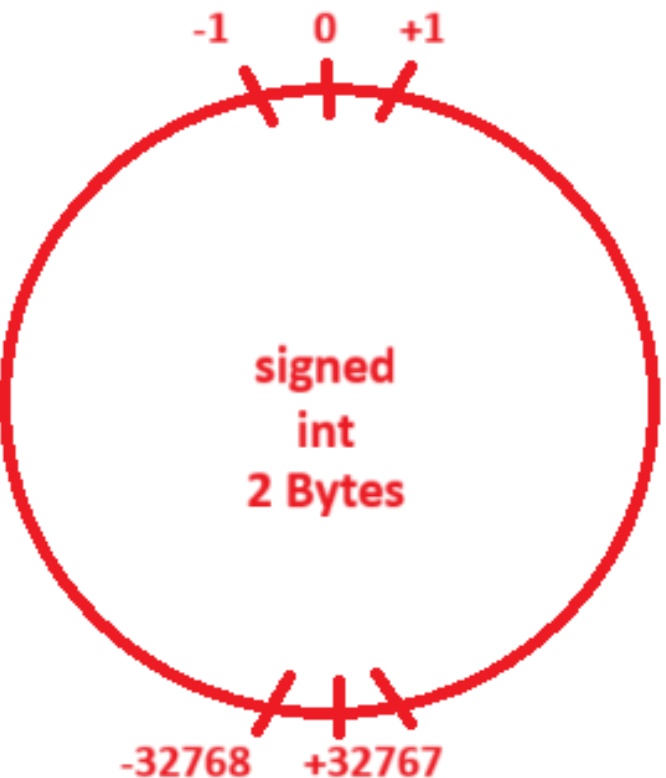
- Integral data types store whole numbers (no fractional part).
- Common types:
 - char → 1 byte
 - short int → 2 bytes
 - int → 2 or 4 bytes (depends on compiler)
 - long int → 4 or 8 bytes
 - long long int → 8 bytes
- Can be **signed** (positive & negative) or **unsigned** (only positive).

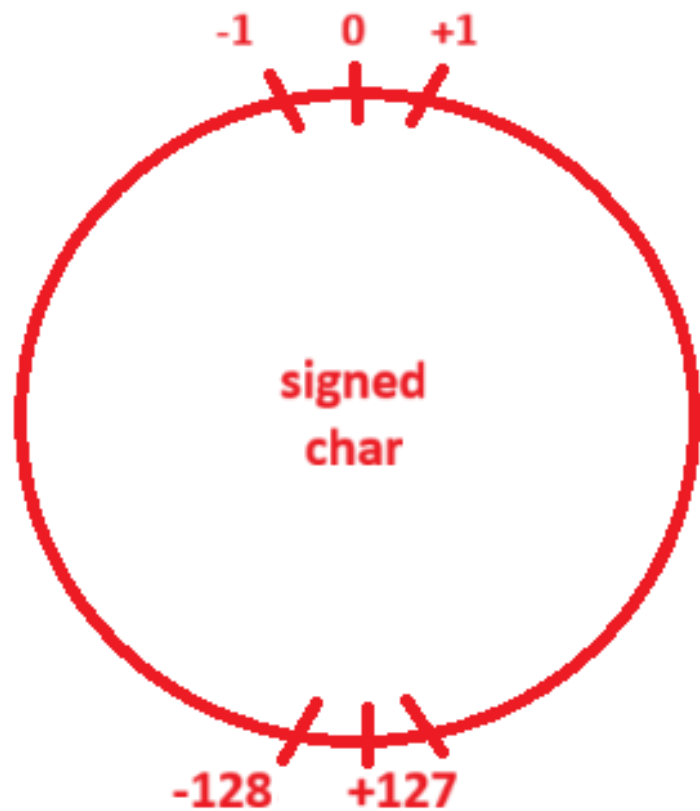
Cyclic Nature of Integral Types

When a variable exceeds its maximum value, it wraps around to minimum value.

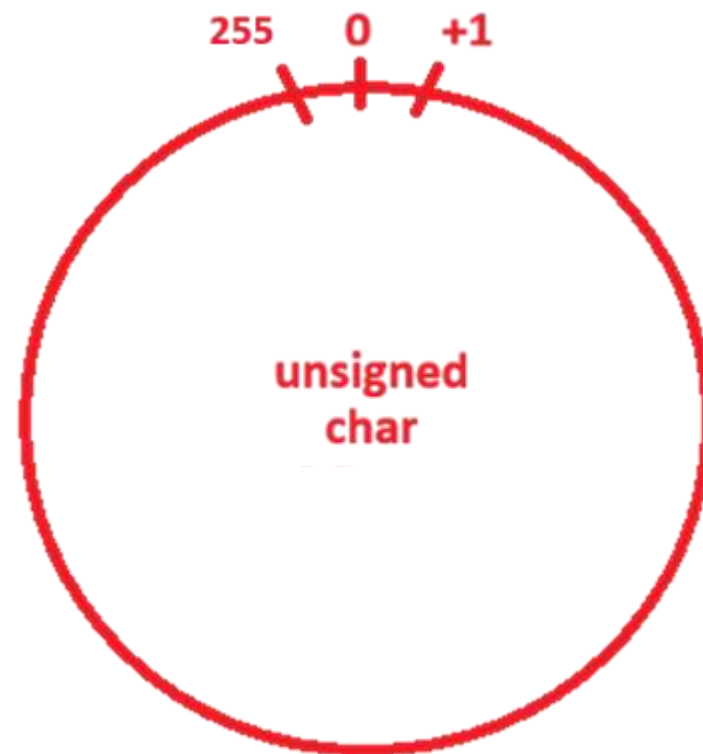
Example:

```
unsigned char x = 255;  
x = x + 1; // x becomes 0  
signed char y = 127;  
y = y + 1; // y becomes -128
```





s-char
130 →
s-char
250 →
u-char
-6 →



Type Conversion in Expressions

Automatic type promotion: Smaller types are converted to larger type before operation.

Rules:

char → int → long → float → double

Example:

char a = 100;

int b = 300;

int c = a + b; // a promoted to int, c = 400

Key Points

- Integral types store whole numbers, signed or unsigned.
- Cyclic nature occurs when values exceed type limits.
- Type conversion ensures safe arithmetic between different types.
- Use casting to avoid unexpected results.

```
main ()  
{  
    int a = 300 * 300 / 300;  
    printf("%d", a);  
}
```

```
main ()  
{  
    char a = 250;  
    int b;  
    b= a + !a + ~a + ++a;  
    printf("%d",b);  
}
```

GATE CSE 2017

Consider the following C program.

```
#include<stdio.h>
#include<string.h>

void printlength(char *s, char *t) {
    unsigned int c=0;
    int len = ((strlen(s) - strlen(t)) > c) ? strlen(s) : strlen(t);
    printf("%d\n", len);
}

void main() {
    char *x = "abc";
    char *y = "defgh";
    printlength(x,y);
}
```

Recall that *strlen* is defined in *string.h* as returning a value of type *size_t*, which is an unsigned int. The output of the program is _____.

printf("%d",3-5 < 0u);

BREAK & CONTINUE

break Keyword in C

- Used to terminate a loop or switch immediately.
- Control jumps outside the loop/switch.
- Commonly used in:
 - switch-case (to exit after one case)
 - loops (to stop early on a condition)

continue Keyword in C

- Used to skip current iteration of a loop.
- Control jumps to the next iteration of the loop.
- Commonly used to avoid executing some part of loop body for specific cases.

Context	break	continue
if statement	✗ Not valid (only works inside loop/switch)	✗ Not valid
switch-case	✓ Valid → exits switch	✗ Not valid
for loop	✓ Exits loop immediately	✓ Skips to next iteration
while loop	✓ Exits loop immediately	✓ Skips to next iteration
do-while loop	✓ Exits loop immediately	✓ Skips to next iteration

IF ELSE

if Statement

Syntax:

```
if (condition) {  
    // code executes if condition is true  
}
```

- Executes block only if condition is true.
- If false → skips the block

if-else Statement

Syntax:

```
if (condition) {  
    // executes if condition is true  
} else {  
    // executes if condition is false  
}
```

- Provides two-way decision making.
- Exactly one block executes.

Nested if

- An if inside another if.

Syntax:

```
if (condition1) {  
    if (condition2) {  
        // executes only if both are true  
    }  
}
```

- Used for hierarchical checking.

Multiple if-else (Ladder)

Syntax:

```
if (condition1) {  
    // block1  
} else if (condition2) {  
    // block2  
} else if (condition3) {  
    // block3  
} else {  
    // default block  
}
```

- Checks multiple conditions sequentially.
- First true condition's block executes.
- If none true → final else executes.

```
1. main()
{
    int i = 3;
    if (i == 3)
        printf("Hello");
}
```

```
2. main()
{
    int a = -10;
    if (a != 10)
        printf("hi");
    else
        printf("bye");
}
```

```
3. main()
{
    int x = -1;
    if (x == -1)
    {
        pf("hi");
    }
    else
        pf("bye");
    pf("hello");
}
```

```
4. main()
{
    int a = 1;
    if (a != 1)
        printf("hi");
        printf("hello");
    else
        printf("bye");
}
```

```
5. main()
{
    int x = 10;
    if (x >= 5);
        printf("bye");
}
```

```
6. main()
{
    int x = 10;
    if (x >= 5);
        printf("hi");
    else
        printf("bye");
}
```

```
7. main()
{
    float x = 3.14;
    if (x < 3.14)
        printf("hi");
    else
        printf("bye");
}
```

```
8. main()
{
    int x = 0;
    if (x = -1)
        printf("hi");
    else
        printf("hello");
}
```

```
9. int main()
{
    float x = 3.14;
    if (x == 3.14f)
        printf("hi");
    else
        printf("bye");
    return 0;
}
```

```
10. main()
{
    if (!printf("hi"))
        if (printf("bye"));
}
```

```
11. main()
{
    signed int i = -10;
    unsigned int j = -10u;

    if (i == j)
        printf("hi");
    else
        printf("bye");
}
```

SWITCH

- Multi-way decision making statement.
- Works like if-else ladder, but cleaner.

Syntax:

```
switch(expression) {  
    case value1:  
        // statements  
    case value2:  
        // statements  
    ...  
    default:  
        // executes if no case matches  
}
```

Features of switch-case

- Expression must evaluate to int or char (no float/double).
- Each case must have a constant or literal value.
- break; prevents fall-through to next case.
- default case is optional.

Feature	switch-case	if-else
Data type supported	int, char	Any (int, float, string...)
Readability	Cleaner for multiple choices	Gets messy with many else
Range checking	✗ Not supported	✓ Supported
Default block	Optional	else is optional

(i) main()

```
{  
    int i = 3;  
  
    switch(i)  
    {  
        case 1: pf("hi");  
        case 2: pf("hey");  
        case 3: pf("hello");  
        default: pf("bye");  
    }  
}
```

(ii) main()

```
{  
    int i = 5;  
  
    switch(i)  
    {  
        case 3: pf("hi");  
        default: pf("hey");  
        case 2: pf("hello");  
        case 1: pf("bye");  
    }  
}
```

```
(iii) main()
{
    int i = 5;
    switch(i)
    {
        printf("hello");
        case 1: printf("hi1");
        case 2: printf("hi2");
        default: printf("default");
        case 3: printf("hi3");
    }
}
```

```
(iv) main()
{
    int i = 6;
    switch(i)
    {
        case 3 << 2: printf("hi");
        case -1: printf("hey");
        case 5 | 2: printf("hello");
        default: printf("bye");
    }
}
```

Q Consider the following C program (**Gate-2017**) (**2 Marks**)

```
#include<stdio.h>
```

```
int main ()
```

```
{
```

```
    int m = 10;
```

```
    int n, n1;
```

```
    n = ++m;
```

```
    n1 = m++;
```

```
    n--;
```

```
    --n1;
```

```
    n -= n1;
```

```
    printf ("%d", n);
```

```
    return 0;
```

```
}
```

The output of the program is _____

Q Consider the following ANSI C program. **(GATE - 2021) (2 Marks)**

```
#include <stdio.h>

int main()
{
    int i, j, count;
    count=0;
    i=0;
    for (j=-3; j<=3; j++)
    {
        if (( j >= 0) && (i++))
            count = count + j;
    }
    count = count + i;
    printf("%d", count);
    return 0;
}
```

Which one of the following options is correct?

- a)** The program will not compile successfully
- b)** The program will compile successfully and output 10 when executed
- c)** The program will compile successfully and output 8 when executed
- d)** The program will compile successfully and output 13 when executed

Q Consider the following C program: **(GATE - 2019) (1 Marks)**

```
#include <stdio.h>

int main ()
{
    float sum = 0.0, j = 1.0, i = 2.0;
    while (i / j > 0.0625)
    {
        j = j + j;
        printf ("%f\n", sum);
    }
    return 0;
}
```

The number of times variable sum will be printed
When the above program is executed is

Q. The output of the given C code segment is _____. (Answer in integer) **(Gate 2025)**

```
int x=126,y=105;
```

```
do {
```

```
    if(x>y) x=x-y;
```

```
    else y=y-x;
```

```
} while(x!=y);
```

```
printf("%d",x);
```


Q Consider the following C program: **(GATE - 2015) (1 Marks)**

```
#include <stdio.h>

int main ()
{
    int i, j, k = 0;
    j = 2 * 3 / 4 + 2.0 / 5 + 8 / 5;
    k -= --j;
    for (i = 0; i < 5; i++)
    {
        switch (i + k)
        {
            case 1:
            case 2: printf ("%d", i + k);
            case 3: printf ("%d", i + k);
            default: printf ("%d", i + k);
        }
    }
    return 0;
}
```

The number of times printf statement is executed is _____.

- a) 8
- b) 9
- c) 10
- d) 11

Q What will be the output of the following C program segment? **(GATE - 2012) (2 Marks)**

```
char inchar = 'A';  
switch (inchar)  
{  
    case 'A': printf ("choice A \ n");  
    case 'B': printf ("choice B ");  
    case 'C':  
    case 'D':  
    case 'E':  
    default: printf ("No Choice");  
}
```

(A) No choice

(B) Choice A

(C) Choice A

Choice B No choice

(D) Program gives no output as it is erroneous

WHILE LOOP

- A loop is used to execute a block of code repeatedly.
- Reduces code repetition.
- Types of loops in C:
 1. Entry-controlled loops: for, while
 2. Exit-controlled loop: do-while

while Loop

Syntax:

```
while(condition) {  
    // statements  
}
```

- Entry-controlled loop.
- Used when number of iterations not fixed.

FOR

for Loop

Syntax:

```
for(initialization; condition; iteration) {  
    // statements  
}
```

- Best when number of iterations is known.

DO-WHILE

do-while Loop

Syntax:

```
do {  
    // statements  
} while(condition);
```

- Exit-controlled loop (executes at least once).

Feature	for loop	while loop	do-while loop
Condition checked	Before execution	Before execution	After execution
Execution guarantee	✗ Not guaranteed	✗ Not guaranteed	✓ At least once
Best use case	Fixed iterations	Unknown iterations	Menu-driven tasks

Q. The output of the given C code segment is _____. (Answer in integer) **(Gate 2025)**

```
int x=126,y=105;
```

```
do {
```

```
    if(x>y) x=x-y;
```

```
    else y=y-x;
```

```
} while(x!=y);
```

```
printf("%d",x);
```

ARRAYS & POINTERS

Contiguous Memory Allocation

- Array elements are stored in contiguous memory blocks. Means, if you skip/jump from one block you will reach the next block

Syntax: `type arr_name[size];`

Example:

```
int arr[5] = {10, 20, 30, 40, 50};
```

Representation:

arr	10	20	30	40	50
	0	1	2	3	4
	100	102	104	106	108

Homogeneous Data Structure

- All elements must be of the same data type.

Example:

```
int arr[3] = {1, 2, 3}; // All integers
```

```
char arr[3] = {'a', 'b', 'c'}; // All characters
```

Diagram:

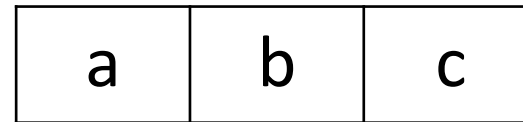


0

1

2

int type



0

1

2

char type

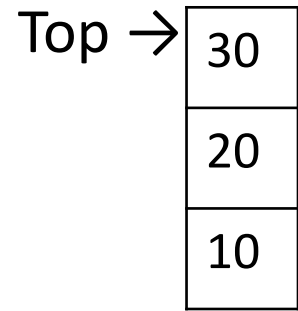
Implementation of Other Structures

- Arrays are used to implement Stacks and Queues.

Example (Stack using array):

stack[5] = {10, 20, 30}

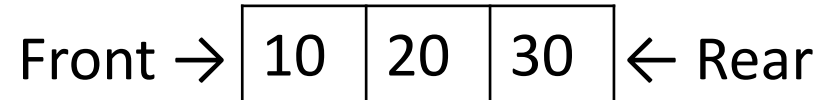
Diagram (Stack - LIFO):



Example (Queue using array):

queue[5] = {10, 20, 30}

Diagram (Queue - FIFO):



Array Indexing

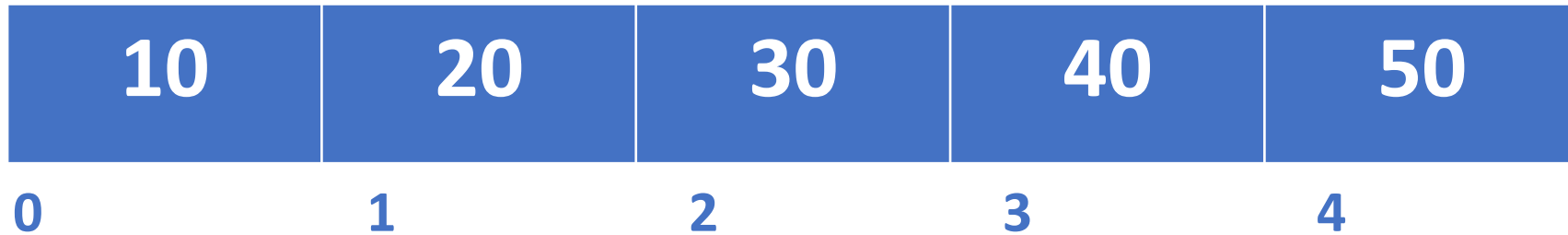
- Array indices in C always start from 0 and go up to size – 1.
- This is because index represents offset from the base address.

Example:

```
int arr[5] = {10, 20, 30, 40, 50};
```

- Valid indices: 0 → 4 (since size = 5)
- arr[0] = 10
- arr[4] = 50

Diagram:



Size Must Be Declared and by default contains garbage values

- Size must be fixed at declaration.

Example:

`int arr[5];` // Valid

`int arr[];` // Invalid (size missing)

Diagram:

arr[5] →

G	G	G	G	G
0	1	2	3	4

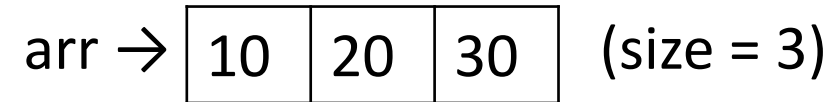
Size Not Specified → Initialization Mandatory

- If you don't specify array size, you must initialize it.
- The size becomes equal to the number of initialized elements.

Example 1:

```
int arr[] = {10, 20, 30}; // size = 3
```

Diagram:



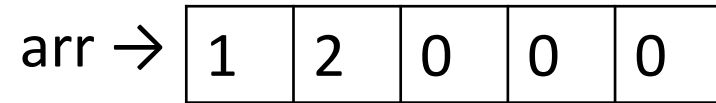
Partial Initialization → Remaining Elements = 0

- If you initialize only some elements, remaining are automatically 0.

Example 1:

```
int arr[5] = {1, 2};
```

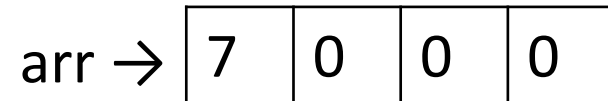
Diagram:



Example 2:

```
int arr[4] = {7}; // only first initialized
```

Diagram:



Introduction to Pointers

- A pointer is a variable that stores the address of another variable.
- Provides direct access to memory.

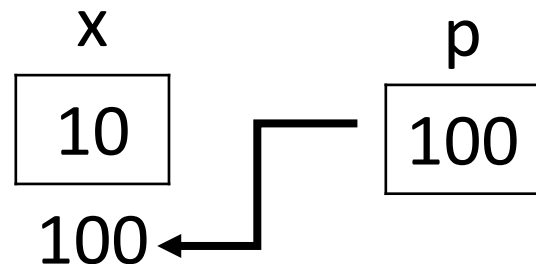
Syntax: `datatype *ptr;`

Pointer Declaration & Initialization

```
int x = 10;  
int *p;    // pointer making  
p = &x;    // assigned with address of x
```

p → stores address of x.

*p → dereferencing → gives value stored at address.



Relationship Between Array and Pointer

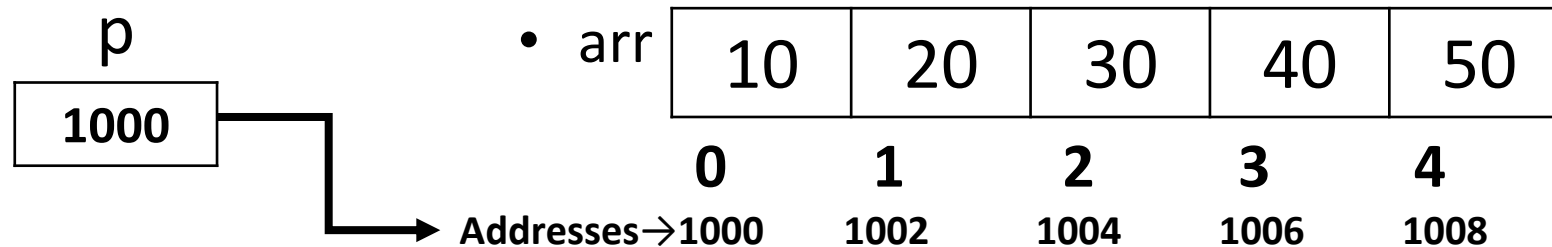
- In C, array name acts like a pointer to its first element.

Example:

```
int arr[5] = {10, 20, 30, 40, 50};  
int *p = arr;
```

Here:

- arr → address of arr[0]
- p → also points to arr[0]



Address Representation

Suppose base address = 1000 (for arr[0])

If int takes 2 bytes:

$$\text{Loc(arr[index])} = \text{Base Address} \pm \text{index} * \text{size_of_datatype/Block Size}$$

Element	Expression	Address Calculation	Address	Values
arr[0]	*(arr + 0)	*(1000 + (0*2))	1000	10
arr[1]	*(arr + 1)	*(1000 + (1*2))	1002	20
arr[2]	*(arr + 2)	*(1000 + (2*2))	1004	30
arr[3]	*(arr + 3)	*(1000 + (3*2))	1006	40
arr[4]	*(arr + 4)	*(1000 + (4*2))	1008	50

Key Points

- Array name (arr) = constant pointer → cannot be reassigned.

`arr = arr + 1; // Error`

- Pointer variable (p) = can be reassigned.

`p = p + 1; // Now points to next element`

Important Points:

- Array name = pointer to first element.
- Array indexing and pointer arithmetic are equivalent.
- Address calculation → $\text{Base} + (i * \text{size_of_datatype})$.
- Array name is a constant pointer, pointer variable is modifiable.

Address Operations

1. Pointer Addition (Address + Constant)

You can add a constant to a pointer. The pointer is incremented by multiple of the size of its data type.

Syntax: `ptr = ptr + n;`

Example:

```
int arr[5] = {10, 20, 30, 40, 50};  
  
int *p = arr;  
  
p = p + 2; //p now points to arr[2], i.e., address of 30
```

2. Subtraction (Address - Constant)

You can subtract a constant from a pointer. The pointer is moved backward by multiple of the size of its data type.

Syntax: `ptr = ptr - n;`

Example: `int arr[5] = {10, 20, 30, 40, 50};`

`int *p = &arr[3];`

`p = p - 1; // p now points to arr[2], i.e., address of 30`

3. Pointer Difference (Address - Address)

You can find the difference between two pointers that point to elements of the same array.

Syntax: `ptr1 - ptr2;`

Explanation: The result is the number of elements between the two pointers, not the byte difference. If ptr1 points to arr[3] and ptr2 points to arr[1], ptr1 - ptr2 will be 2 (i.e., &arr[3] - &arr[1]).

Example:

```
int arr[5] = {10, 20, 30, 40, 50};  
int *p1 = &arr[4];  
int *p2 = &arr[1];  
printf("%d", p1 - p2); // Output: 3
```

p1 - p2 gives 3 because there are 3 elements between arr[1] and arr[4].

`int a[] = {10, 20, 30, 40, 50, 60};`

`a =`

`&a =`

`*a =`

`**a =`

`&a[0] =`

`a+1 =`

`*(a+1) =`

`a[0] =`

10	20	30	40	50	60
0	1	2	3	4	5

`a[8] =`

`a-2 =`

`a[-2] =`

`*(a-2) =`

`&(a+1) =`

`&a+1 =`

```
int a[2][3] = { {10,20,30}, {40,50,60} };
```

a =

&a[0] =

&a =

a+1 =

*a =

*(a+1) =

**a =

**(a+1) =

***a =

**a+1 =

```
int a[2][3] = { {10,20,30}, {40,50,60} };
```

`a[1] =`

10	20	30	40	50	60
0	1	2	3	4	5
100	102	104	106	108	110

`a[1]+2 =`

0
100

1
106

`*a[1]+2 =`

`a[1][2] =`

`*(*a+1)+2 =`

`*(a[0]+5) =`

```
int a[2][3] = { {10,20,30}, {40,50,60} };
```

10	20	30	40	50	60
0	1	2	3	4	5
100	102	104	106	108	110

`*(a[0]+10) =`

0
100

1
106

`a[1]-2 =`

`*(a[1]-3) =`

`&a+1 =`

`*(*(&a+1)-5) =`

```
int a[2][3] = { {10,20,30}, {40,50,60} };
```

10	20	30	40	50	60
0	1	2	3	4	5
100	102	104	106	108	110
0			1		
100			106		

`*(*(a[1]+1) =`

`a[-2]+6 =`

`*a[-3]+10 =`

`*( *a-3+10) =`

```
main()
{
    int a[] = {10, 20, 30, 40, 50, 60};
    int *p[] = {a, a+1, a+2, a+3, a+4, a+5};
    int **ptr = p;

    ptr++;
    printf("%d %d %d", ptr - p, *ptr - a, **ptr);

    *ptr++;
    printf("%d %d %d", ptr - p, *ptr - a, **ptr);

    ++*ptr;
    printf("%d %d %d", ptr - p, *ptr - a, **ptr);

    ++**ptr;
    printf("%d %d %d", ptr - p, *ptr - a, **ptr);
}
```

Q Consider the following C program: **(GATE - 2019) (2 Marks)**

```
#include<stdio.h>
int main ()
{
    int arr [] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 5}, *ip = arr + 4;
    printf ("%d\n", ip[1]);
    return 0;
}
```

The number that will be displayed on execution of the program is _____.

Q Consider the following C program. **(GATE - 2015) (1 Marks)**

```
#include<stdio.h>
```

```
int main ()
```

```
{
```

```
    static int a[] = {10, 20, 30, 40, 50};
```

```
    static int *p[] = {a, a+3, a+4, a+1, a+2};
```

```
    int **ptr = p;
```

```
    ptr++;
```

```
    printf ("%d%d", ptr - p, **ptr);
```

```
}
```

The output of the program is _____

a) 140

b) 120

c) 100

d) 40

Q. What is the output of the following C program? (Gate 2024 CS) (2 Marks) (MCQ)

```
#include<stdio.h>
int main()
{
    double a[2]={20.0, 25.0}, *p, *q;
    p = a;
    q = p + 1;
    printf("%d,%d", (int)(q - p), (int)(*q - *p));
    return 0;
}
```

(a) 4,8

(b) 1,5

(c) 8,5

(d) 1,8

Ans: B

```
main()
{
    int a[] = {50, 40, 30, 20, 10};
    int *p[] = {a+2, a+3, a+1, a, a+4};
    int **ptr;
    ptr = p;

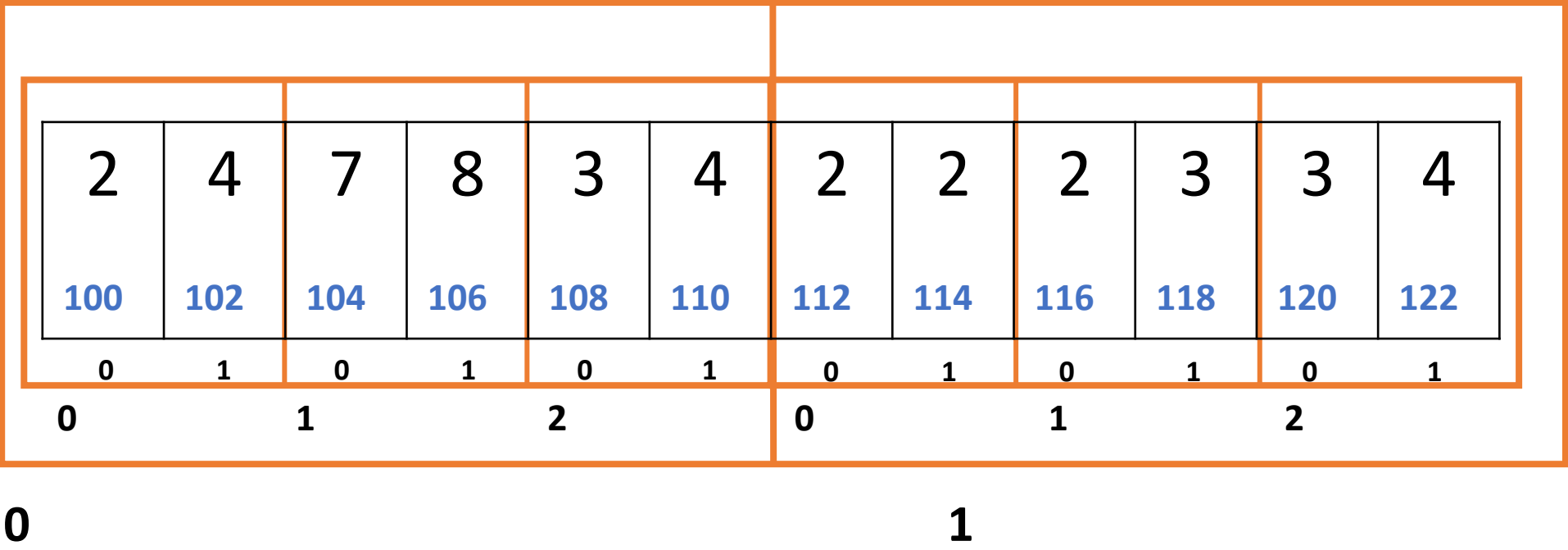
    **ptr++;
    printf("%d %d %d", ptr - p, *ptr - a, **ptr);

    *++*ptr;
    printf("%d %d %d", ptr - p, *ptr - a, **ptr);

    ++**ptr;
    printf("%d %d %d", ptr - p, *ptr - a, **ptr);
}
```

```
main()
{
    int a[2][3][2] = {2, 4, 7, 8, 3, 4, 2, 2, 2, 3, 3, 4};

    printf(a, *a, **a, ***a);
    printf(a+1, *a+1, **a+1, ***a+1);
    printf(a+7, a[5]+7, *a[5]+7, **a[5]+7);
}
```



Q What is printed by the following ANSI C program? **(GATE 2022) (1 MARKS)**

```
#include<stdio.h>

int main(int argc, char *argv[])
{
    int x = 1, z[2] = {10, 11};
    int *p = NULL;
    p = &x;
    *p = 10;
    p = &z[1];
    *(&z[0] + 1) += 3;
    printf("%d, %d, %d\n", x, z[0], z[1]);
    return 0;
}
```

- (A)** 1, 10, 11
- (B)** 1, 10, 14
- (C)** 10, 14, 11
- (D)** 10, 10, 14

Q Consider the following C program: **(GATE - 2020) (1 Marks)**

```
#include <stdio.h>

int main ()
{
    int a[4] [5] = {{1, 2, 3, 4, 5},
                    {6, 7,8, 9, 10},
                    {11, 12, 13, 14, 15},
                    {16, 17,18, 19, 20}};

    printf("%d\n", *(*a+**a+2)+3);
    return(0);
}
```

Q. Consider the following C program: (Gate 2025)

```
#include<stdio.h>
```

```
int main(){
```

```
    int a;
```

```
    int arr[5] = {30,50,10};
```

```
    int *ptr;
```

```
    ptr = &arr[0] + 1;
```

```
    a = *ptr;
```

```
    (*ptr)++;
```

```
    ptr++;
```

```
    printf("%d", a + (*ptr) + arr[1]);
```

```
    return 0;
```

```
}
```

The output of the above program is _____. (Answer in integer)

Q What is printed by the following ANSI C program? **(GATE 2022) (2 MARKS)**

```
#include<stdio.h>

int main(int argc, char *argv[])
{
    int a[3][3][3]={{1, 2, 3, 4, 5, 6, 7, 8, 9},
                    {10, 11, 12, 13, 14, 15, 16, 17, 18},
                    {19, 20, 21, 22, 23, 24, 25, 26, 27}};
    int i = 0, j = 0, k = 0;
    for( i = 0; i < 3; i++ )
    {
        for(k = 0; k < 3; k++ )
            printf("%d ", a[i][j][k]);
        printf("\n");
    }
    return 0;
}
```

(A) 1 2 3
10 11 12
19 20 21

(B) 1 4 7
10 13 16
19 22 25

(C) 1 2 3
4 5 6
7 8 9

(D) 1 2 3
13 14 15
25 26 27

CHARACTER POINTER

- A character pointer is a pointer that stores the address of a character variable.

Syntax: `char *p;`

- Used for handling characters and strings efficiently.

Character Pointer Example

```
char ch = 'A';
```

```
char *p = &ch;
```

```
printf("%c", *p); // Output: A
```

p → holds address of ch.

*p → dereferences pointer → gives 'A'.

Character Pointer and Strings

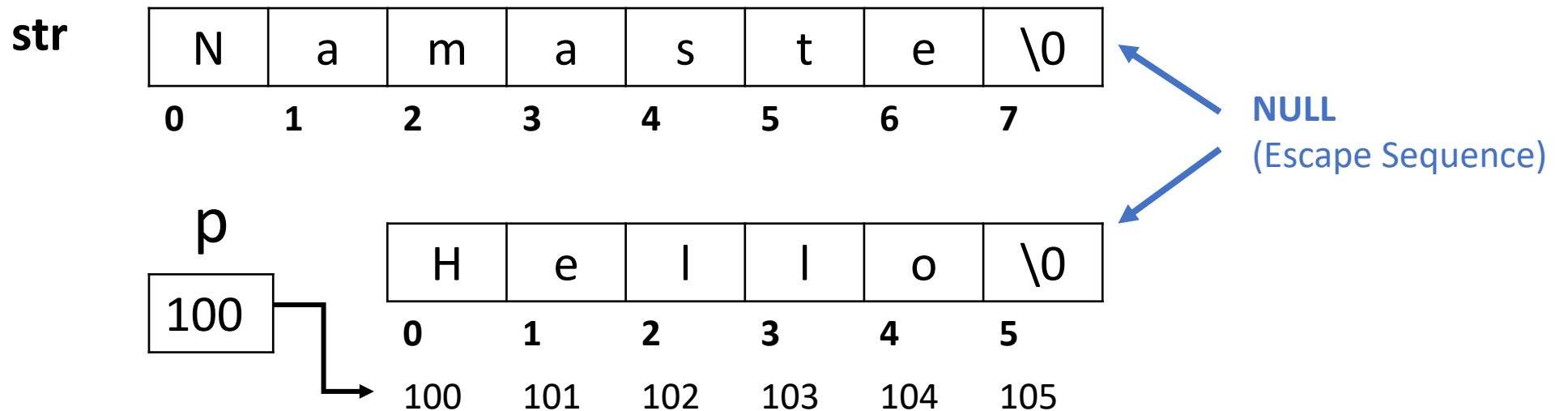
- A string is an array of characters, so it can be represented by a character pointer.

```
char str[] = "Namaste";
```

```
char *p = "Hello";
```

```
printf("%s %s", p, str); // Output: Hello Namaste
```

- p points to the first character of the string.
- Pointer moves sequentially through the string.



```
main()
{
    char *s[] = {"ice", "cream", "cone", "please"};
    char **ptr[] = {s+3, s+2, s+1, s};
    char ***p = ptr;

    printf("%s", **++p);
    printf("%s", *--*++p + 1);
}
```

```
main()
{
    char *c[] = {"ENTER", "NEW", "POINT", "FIRST"};
    char **cp[] = {c+3, c+2, c+1, c};
    char ***cpp = cp;

    printf("%s", **++cpp);           // (1)
    printf("%s", *--*++cpp+3);       // (2)
    printf("%s", *cpp[-2]+3);        // (3)
    printf("%s", cpp[-2][-1]+1);     // (4)
}
```

Q. What is the output printed by statement (1) and statement (2) in the above program?

- a) POINT, ER
- b) POINT, EW
- c) NEW, ER
- d) FIRST, EW

```
main()
{
    char *c[] = {"ENTER", "NEW", "POINT", "FIRST"};
    char **cp[] = {c+3, c+2, c+1, c};
    char ***cpp = cp;

    printf("%s", **++cpp);           // (1)
    printf("%s", *--*++cpp+3);       // (2)
    printf("%s", *cpp[-2]+3);        // (3)
    printf("%s", cpp[-2][-1]+1);     // (4)
}
```

Q. What is the output printed by statement (3) and statement (4) in the above program?

- a) POINT, ER
- b) ST, OINT
- c) NEW, ER
- d) POINT, EW

Q Consider the following program (**GATE - 2017**) (**2 Marks**)

```
#include<stdio.h>
#include<string.h>
int main ()
{
    char * c = "GATECSIT2017";
    char *p = c;
    printf ("%d", (int)strlen (c + 2[p] - 6[p] - 1));
    return 0;
}
```

The Output of the following program is_____

Q Consider the following C program segment. **(GATE - 2015) (1 Marks)**

```
# include <stdio.h>
```

```
int main ()
```

```
{
```

```
    char s1[7] = "1234", *p;
```

```
    p = s1 + 2;
```

```
    *p = '0';
```

```
    printf ("%s", s1);
```

```
}
```

What will be printed by the program?

a) 12

b) 120400

c) 1204

d) 1034

Q What does the following fragment of C-program print? **(Gate-2011) (1 Marks)**

```
char c[] = "GATE2011";  
char *p = c;  
printf("%s", p + p[3] - p[1]) ;
```

(A) GATE2011

(B) E2011

(C) 2011

(D) 011

FUNCTIONS

- Functions are blocks of code that perform a specific task.
- They help in code modularity, reusability, and organization.
- A function in C can take parameters and return values.

Syntax:

```
return_type function_name(parameters) {  
    // function body  
}
```

Important Point: Whenever we put void as a return_type the function always return something **undefined** (undefined by the user)

PARAMETER PASSING TECHNIQUES

1. Call by Value

A copy of the argument is passed to the function. Changes to the parameter inside the function do not affect the original variable.

2. Call by Address

This passes the address of the variable. Allows the function to modify the value directly, since it has access to the original address.

3. Call by Reference

Address of the variable is passed, so the function works directly with the original variable. Changes made to the parameter affect the original variable. Achieved using pointers.

```
void swap(int a, int b) {  
    int temp;  
    temp = a;  
    a = b;  
    b = temp;  
}
```

```
int main() {  
    int x = 5, y = 10;  
    swap(x, y);  
    printf("x = %d, y = %d", x, y);  
}
```

```
void swap(int *a, int *b) {  
    int temp;  
    temp = *a;  
    *a = *b;  
    *b = temp;  
}  
  
int main() {  
    int x = 5, y = 10;  
    swap(&x, &y);  
    printf("x = %d, y = %d", x, y);  
}
```

```
void swap(int &a, int &b) {  
    int temp;  
    temp = a;  
    a = b;  
    b = temp;  
}
```

```
int main() {  
    int x = 5, y = 10;  
    swap(x, y);  
    printf("x = %d, y = %d", x, y);  
}
```

Q Consider the following C program:

```
#include <stdio.h>
```

```
void fun1(char *s1, char *s2)  
{  
  
    char *temp;  
  
    temp = s1;  
  
    s1 = s2;  
  
    s2 = temp;  
  
}
```

```
void fun2(char **s1, char **s2)  
{  
  
    char *temp;  
  
    temp = *s1;  
  
    *s1 = *s2;  
  
    *s2 = temp;  
  
}
```

```
int main ()  
{  
  
    char *str1 = "Hi", *str2 = "Bye";  
  
    fun1(str1, str2);  
  
    printf ("%s %s", str1, str2);  
  
    fun2(&str1, &str2);  
  
    printf ("%s %s", str1, str2);  
  
    return 0;  
  
}
```

The output of the program above is **(GATE - 2018) (2 Marks)**

- a) Hi Bye Bye Hi
- c) Bye Hi Hi Bye

- b) Hi Bye Hi Bye
- d) Bye Hi Bye Hi

Q Consider the following C program. (GATE - 2016) (1 Marks)

```
#include<stdio.h>

void mystery (int *ptr a, int *ptr b)
{
    int *temp;
    temp = ptr b;
    ptr b = ptr a;
    ptr a = temp;
}

int main ()
{
    int a=2016, b=0, c=4, d=42;
    mystery (&a, &b);
    if (a < c)
        mystery (&c, &a);
    mystery (&a, &d);
    printf ("%dn", a);
}
```

The output of the program

Q The value printed by the following program is **(GATE - 2016) (1 Marks)**

```
void f (int* p, int m)
{
    m = m + 5;
    *p = *p + m;
    return;
}
```

```
void main ()
{
    int i=5, j=10;
    f (&i, j);
    printf ("%d", i + j);
}
```

a) 10

b) 20

c) 30

d) 40

Q The output of the following C program is _____. **(GATE - 2015) (1 Marks)**

```
void f1 (int a, int b)
{
    int c;
    c=a; a=b; b=c;
}

void f2 (int *a, int *b)
{
    int c;
    c=*a; *a=*b;*b=c;
}

int main ()
{
    int a=4, b=5, c=6;
    f1(a, b);
    f2(&b, &c);
    printf ("%d", c-a-b);
    return 0;
}
```

a) -5

b) -4

c) 5

d) 3

Q What does the following program print? **(Gate - 2010) (1 Marks)**

```
#include<stdio.h>
void f (int *p, int *q)
{
    p = q;
    *p = 2;
}
int i = 0, j = 1;
int main ()
{
    f (&i, &j);
    printf ("%d %d \n", i, j);
    getchar ();
    return 0;
}
```

(A) 2 2

(B) 2 1

(C) 0 1

(D) 0 2

Q Let swap () be a function that swaps two elements using their addresses. Consider the following C function.

(GATE - 2015) (2 Marks)

```
void fun (int arr [], int n)
{
    for (int i = 0; i < n; i+=2)
    {
        if (i>0 && arr[i-1] > arr[i])
            swap(&arr[i], &arr[i-1]);

        if (i<n-1 && arr[i] < arr[i+1])
            swap(&arr[i], &arr [i + 1]);
    }
}
```

If an array {10, 20, 30, 40, 50, 60, 70, 80} is passed to the function, the array is changed to

(A) {20, 10, 40, 30, 60, 50, 80, 70}

(B) {10, 30, 20, 40, 60, 50, 80, 70}

(C) {10, 20, 30, 40, 50, 60, 70, 80}

(D) {80, 70, 60, 50, 40, 30, 20, 10}

Q. Consider the following C program: **(Gate 2025)**

```
#include <stdio.h>

void stringcopy (char *, char *);

int main() {
    char a[30] = "@#Hello World!";
    stringcopy(a, a+2);
    printf("%s\n",a);
    return 0;
}

void stringcopy(char *s, char *t) {
    while (*t)
        *s++=*t++;
}
```

Which one of the following will be the output of the program?

- A)** @#Hello World!
- B)** Hello World!
- C)** ello World!
- D)** Hello World!d!

Q. The output of the given C program is _____. (Answer in integer) **(Gate 2025)**

```
#include <stdio.h>

void foo(int *p, int x) {
    *p=x;
}

int main(){
    int *z;
    int a = 20, b = 25;
    z = &a;
    foo(z, b);
    printf("%d", a);
    return 0;
}
```

STRUCTURES

- A structure is a user-defined datatype in C.
- Groups variables of different datatypes under one name.
- Each member has separate memory allocation.

Syntax:

```
struct Student {  
    int var1;  
    char var2;  
    float var3;  
};
```

Example:

```
struct Student {  
    int roll;  
    char name[20];  
    float marks;  
};  
struct Student s1 = {101, "Ravi", 87.5};  
printf("%d %s %f", s1.roll, s1.name, s1.marks);
```

Memory in Structure

Memory = sum of all members' sizes (with possible padding).

Example:

```
struct Test {  
    int x;    // 4 bytes  
    char y;   // 1 byte  
    float z;  // 4 bytes  
};
```

Total size ≥ 9 bytes (depends on compiler, may use padding).

Union in C

- A union is also a user-defined datatype.
- Groups variables of different datatypes, but shares the same memory location.
- Only one member can hold a value at a time.

Example:

```
union Data {  
    int i;  
    float f;  
    char str[20];  
};
```

```
union Data d1; //Total size = 20 bytes.  
d1.i = 10;  
printf("%d", d1.i);
```

```
d1.f = 22.5;  
printf("%.2f", d1.f); // overwrites previous value
```

```
main()
{
    static struct S1
    {
        char c[4], *s;
    } s1 = {"abc", "def"};

    static struct S2
    {
        char *cp;
        struct S1 ss1;
    } s2 = {"ghi", "jkl", "mno"};

    printf("%s %s", ++s2.cp, ++s2.ss1.s);
}
```

- a) ghi, mno
- b) hi, mn
- c) hi, no
- d) ghi, no

```

struct S1
{
    char *s;
    int i;
    struct S1 *q;
};

main()
{
    static struct S1 a[] = {
        {"abcd", 1, a+1},
        {"efgh", 2, a+2},
        {"ijkl", 3, a}
    };

    struct S1 *p = a;
    int i;
    printf("%s, %s, %s", a[0].s, p->s, a[2].q->s);
}

```

- a) abcd, abcd, abcd
- b) abcd, abcd, ijkl
- c) abcd, ijkl, abcd
- d) abcd, ijkl, ijkl

```
main()
{
    struct a
    {
        char ch[7];
        char *str;
    };

    struct b
    {
        char *c;
        struct a ss1;
    };

    struct b s2 = {"raipur", "kanpur", "jaipur"};

    printf("%s %s", s2.c, s2.ss1.str);
    printf("%s %s", ++s2.c, ++s2.ss1.str);
}
```

STORAGE CLASSES

- **Storage class** defines scope, lifetime, visibility, and default value of a variable.
- Four main storage classes:
 1. auto
 2. Static
 3. register
 4. extern

auto

- Default storage class for local variables.
- Scope → local to block/function.
- Lifetime → till function ends.
- Default value → garbage

```
void main() {  
    f1();  f1();  f1();  
}  
void f1()  
{  
    auto int i=10;  
    ++i;  
    printf("%d", i);  
}
```

static

- Retains value between multiple function calls. (Visibility → throughout the program)
- Scope → local to block/function (but stored in data segment, not stack).
- Default value → 0.

```
void main() {  
    f1();  f1();  f1();  
}  
void f1()  
{  
    static int i=10;  
    ++i;  
    printf("%d", i);  
}
```

register

- Suggests storing variable in CPU register (faster access).
- Scope → local to block/function.
- Lifetime → till function ends.
- Default value → garbage.

```
void main() {  
    register int i = 40;  
    int *p = &i;  
    ++*p;  
    printf("%d", i);  
}
```

extern/global

- Used to declare a **global variable** defined in another file or scope.
- Scope → global.
- Lifetime → entire program.
- Default value → 0.

```
extern int x; // defined in another file
void main() {
    printf("%d", x);
}
```

Storage Class	Scope	Lifetime	Default Value	Location
auto	Local	Function scope	Garbage	Stack
register	Local	Function scope	Garbage	CPU Reg
static	Local/Global	Entire program	0	Data seg
extern	Global	Entire program	0	Data seg

Q Consider the following C program: **(GATE - 2019) (2 Marks)**

```
#include<stdio.h>
```

```
int r ()  
{  
    int static num=7;  
    return num--;  
}
```

```
int main ()  
{  
    for (r (); r (); r ())  
    {  
        printf ("%d ", r ());  
    }  
    return 0;  
}
```

The output of above C program is _____.

SESSION-7

By Ekagra Sir

Q The output of executing the following C program is _____. **(GATE - 2017) (2 Marks)**

```
#include <stdio.h>
```

```
int total (int v)
```

```
{
```

```
    static int count = 0;
```

```
    while (v)
```

```
    {
```

```
        count += v & 1;
```

```
        v >>= 1;
```

```
    }
```

```
    return count;
```

```
}
```

```
void main ()
```

```
{
```

```
    static int x = 0;
```

```
    int i = 5;
```

```
    for (; i > 0; i--)
```

```
    {
```

```
        x = x + total(i);
```

```
    }
```

```
    printf ("%dn", x) ;
```

```
}
```


Q What will be the output of the following C program? **(GATE - 2016) (1 Marks)**

```
void count (int n)
{
    static int d = 1;
    printf ("%d ", n);
    printf ("%d ", d);
    d++;
    if (n > 1)
        count(n-1);
    printf ("%d ", d);
}
```

```
int main ()
{
    count (3);
}
```

a) 3 1 2 2 1 3 4 4 4

b) 3 1 2 1 1 1 2 2 2

c) 3 1 2 2 1 3 4

d) 3 1 2 1 1 1 2

Q Consider the following C program. The output of the program is _____. (GATE - 2015) (1 Marks)

```
#include <stdio.h>

int f1(void);
int f2(void);
int f3(void);
int x = 10;
int main ()
{
    int x = 1;
    x += f1() + f2() + f3() + f2();
    printf("%d", x);
    return 0;
}
```

```
int f1()
{
    int x = 25;
    x++;
    return x;
}
```

```
int f2( )
{
    static int x = 50;
    x++;
    return x;
}
```

```
int f3( )
{
    x *= 10;
    return x;
}
```

- a) 230
- b) 131
- c) 231
- d) 330

DMA

Dynamic memory allocation allows you to allocate memory at runtime using functions from `stdlib.h`.

- Three main functions:

1. `malloc()`

2. `calloc()`

3. `free()`

malloc()

- `malloc(size_t size)`
- Allocates a block of memory of size bytes.
- Does not initialize the memory.
- Returns a pointer to the allocated memory, or NULL if allocation fails.
- Returns `void*` (an address of newly created memory from heap).

Syntax:

```
type *ptr_name = (type*) malloc(n * sizeof(type));
```

Example:

```
int *ptr = (int*)malloc(10 * sizeof(int)); // allocates space for 10 integers
```

calloc()

- `calloc(size_t n, size_t size)`
- Allocates memory for **n** elements, each of size **size** bytes.
- Initializes all elements to **0**.
- Returns a pointer to the allocated memory or NULL if allocation fails.
- Returns void* (an address of newly created memory from heap).

Syntax: `type *ptr_name = (type*) calloc(n, sizeof(type));`

Example:

```
int *ptr = (int*) calloc(10, sizeof(int)); // allocates and initializes 10 integers to 0
```

Feature	malloc()	calloc()
Initialization	Does not initialize memory	Initializes memory to 0
Function Parameters	Single argument (size)	Two arguments (n, size)
Memory Allocation	Allocates a single block	Allocates memory for multiple elements
Speed	Faster (no initialization)	Slower (due to initialization)

Memory Leak Problem

- Memory leak occurs when dynamically allocated memory is not freed using free().
- This leads to wasted memory, which can cause program crashes or system slowdowns.

Example of Memory Leak:

```
int *ptr = (int*)malloc(5 * sizeof(int));  
// Memory allocated but never freed
```

Solution: Always free dynamically allocated memory after use.

```
free(ptr); // frees the dynamically allocated memory
```

Q Match the following: (Gate-2017) (2 Marks)

(P) <code>static char var;</code>	(i) Sequence of memory locations to store addresses
(Q) <code>m = malloc(10);</code> <code>m = NULL;</code>	(ii) A variable located in data section of memory
(R) <code>char *ptr[10];</code>	(iii) Request to allocate a CPU register to store data
(S) <code>register int var1;</code>	(iv) A lost memory which cannot be freed

(A) P→(ii), Q→(iv), R→(i), S→(iii)

(B) P→(ii), Q→(i), R→(iv), S→(iii)

(C) P→(ii), Q→(iv), R→(iii), S→(i)

(D) P→(iii), Q→(iv), R→(i), S→(ii)

MACROS

Macros in C

- Macros are preprocessor directives that define code replacements before compilation.
- They are defined using the `#define` directive.
- Macros help to increase efficiency, reduce code duplication, and make programs more readable.

Syntax of Macros

`#define MACRO_NAME replacement_code` // #define Formal Actual

Example:

```
#define PI 3.14  
printf("Value of PI: %f", PI);
```

Types of Macros

1. Object-like Macros

- Simple substitution of constants.

Example:

```
#define MAX 100
```

```
int arr[MAX]; // declares an array of size 100
```

2. Function-like Macros

- Macros that accept arguments and return values.

Example:

```
#define SQUARE(x) ((x) * (x))
```

```
int result = SQUARE(5); // result = 25
```

Benefits of Macros

- Faster Execution: Substitutes the code before compilation.
- No Function Call Overhead: No need for function calls, saving execution time.
- Easier Maintenance: Changes in one place (macro definition) affect all occurrences in the program

```
1. #define abc(x) x*x
void main()
{
    int a;
    a = 64/abc(4);
    printf("%d",a);
}
```

2. #define mno pqr

void main()

{

printf("mno");

}

3. #define fun(i, j) i##j

void main()

{

int f1 = 10, f12 = 20;

printf("%d",fun(f1,2));

}

4. #define int char

void main()

{

int a = 10, *p = &a;

*p = 20;

printf("%d %d", sizeof(a), sizeof(p));

}

```
#include<stdio.h>
int main() {
    char far *p =(char far *)0x55550005;
    char far *q =(char far *)0x53332225;
    *p = 25;
    (*p)++;
    printf("%d", *q);
    return 0;
}
```

- (A) 25
- (B) Address
- (C) Garbage
- (D) Compilation Error
- (E) None of above